AI6125 Multi-Agent Systems - Tileworld Multi-Agent System

Group 18

Ananya Singh Twissa Modi Aditya Kumar Goel
Yu Taek Lee Nik Shahrul Nafiz Bin Nik Azhan Máté Tringer

Abstract

In this report we present a six agent system designed for the TileWorld simulation using the MASON framework. In this multi agent benchmarking simulation agents need to work together to explore a partially observable grid, fetch and deliver limited-lifetime tiles while also discovering the initially hidden fuel station. Our approach uses a two-layer architecture with a shared basis including A* pathfinding, fuel-aware planning, memory, and communication protocol; and a specialization layer with a specific role assigned to each agent. Throughout our development phase, our average reward in Config 1 improved from 32.1 to 864.3 and an average reward of 888.6 in Config 2 (for validation) with zero failures, despite the elevated challenges in Config 2. The largest recorded gain came from introducing multi-agent coordination in Phase 3 of our development, while Phase 4 further improved our scores with the introduction of role-specific behavior grounded in the bold-vs-cautious agent framework of Pollack and Ringuette (1990).

Introduction

The Tile World simulation a famous testing environment for agent architectures under resource constraints with partial observability (1). In this environmnet six agents are placed in a bounded grid to cooperate and earn rewards from picking up tiles and delivering them to holes within a fixed lenght experiment.

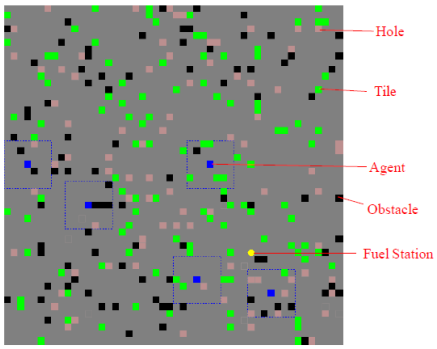

Figure 1: The TileWorld simulation environment.

Copyright © 2026, Association for the Advancement of Artificial Intelligence (www.aaai.org). All rights reserved.

This environment is particullarly challanging because the agents can only see a predifined local area around them while both tiles and holes expiring after a limited number of steps, making timing and up-to-date global information critical. Agent are also subject to a constraint of fuel usage, as every step drains their tank, that can only be refilled at a single (initially hidden) fuelstation.

Based on these constraints it is easy to see how a naive approach - with six independent agents each running the same generic loop - is sub-optimal as it wastes effort through redundant exploration, duplicate targeting, and identical routing. Thus, our goal was to engineer a complex agent architecture, communication and planning protocol to make the six agent behave like a coordinated team while keeping each agent's design tractable and individually testable.

Our solution consists of a *two-layer architecture*. The base layer is shared among all the agents with three implemented classes `SmartTWAgent` (the base agent), `SmartTWAgentMemory` (time-stamped shared memory), and `SmartTWPlanner` (goal-directed, fuel-aware planning). This is then complemented with a role specific layer, to make six specialised agents, with different priorities: fuel discovery, tile collection, delivery reliability, map coverage, high-value routing, and failure prevention.

Background

The TileWorld simulation was introduced by Pollack and Ringuette (1) to provide an experimental to study agent deliberation. Their main objective was to see what the trade-off is between *bold* agents that commit to plans and execute them sequentially without interruption, and *cautious* agents that iteratively reconsider their plan during execution. In their findings they concluded that neither extreme dominates the other, and the optimal balance largely dependent on the rate of change in the environment.

Subsequent work surveyed in (2) confirmed that in high-volatility settings (many objects and/or short lifetimes) cautious behaviour is advantageuous while in lower-volatility environments (sparse, long-lived objects) the committed agent will yield to higher rewards.

In our development we considered these inshights and within the shared layer we defined parameters that depend on the volatility of the environment, that is detected on-the-fly. We also introduced the `SmarterReplanningAgent` with specialist functions as a configurable cautious agent: in higher-volatility con-

ditions it calculates whether a target will survive until arrival and decides to pursue that given target accordingly. In stable conditions it uses a looser threshold and commits longer.

System Architecture

Two-Layer Design

The architecture separates concerns into two layers.

Layer 1 — Shared Foundation. All six agents inherit from `SmartTWAgent` and use the same `SmartTWAgentMemory` and `SmartTWPlanner`. This layer provides: A* pathfinding with obstacle avoidance and fuel awareness; a memory store with per-cell observation timestamps and automatic staleness tracking; a claim system to prevent two agents targeting the same object; runtime environment classification (`isDense()`, `isShortLifetime()`, `isLargeGrid()`); and a full broadcast communication stack.

Layer 2 — Specialist Agents. Each specialist extends `SmartTWAgent` and overrides the `think()` method to insert role-specific decisions before delegating to the shared base logic via `super.think()`. This keeps each specialist focused on its role while retaining all base capabilities unchanged.

Shared Agent Loop

Every simulation step follows a fixed ordering enforced by the MASON scheduler:

1. **Environment step** (ordering 1): new objects are created, expired objects are removed, and the message list is cleared.
2. **Sense and communicate** (ordering 2): each agent observes nearby cells, updates memory, reads incoming messages, and broadcasts its own.
3. **Think and act** (ordering 3): each agent runs its decision logic and executes one action (move, pickup, drop, refuel, or wait).

Within ordering 2, agents run `sense(); communicate();` sequentially in creation order, each reading the message queue before appending to it. A message broadcast by agent N is therefore visible during the same step to agents $N+1$ through 6 (which read after N writes), but not to agents 1 through $N-1$ (which already read before N wrote). By ordering 3, all broadcasts from ordering 2 are in the queue; however, each agent's stored parsed data reflects only the messages that existed when it called `communicate()`, so earlier-ordered agents see later-ordered agents' broadcasts at the following step rather than the current one. In practice this means fuel station discoveries and tile or hole sightings propagate to the full team by the following step at the latest, and to later-ordered agents within the same step.

A* Pathfinding

Navigation is handled by `AstarPathGenerator`, which computes obstacle-avoiding paths using the standard A* algorithm with Manhattan distance as the heuristic. The planner wraps path generation with an affordability check: before committing to a goal, it verifies that the agent has enough fuel to reach the target and return to the fuel station. In short-lifetime environments the planner also rejects targets whose estimated remaining lifetime is shorter than the steps required to arrive.

Memory and Environment Detection

`SmartTWAgentMemory` stores observation timestamps for every seen cell, tile, and hole. These timestamps drive three runtime signals:

- `isShortLifetime()` — true when observed object disappearance rates suggest objects expire in roughly 30 steps or fewer.
- `isDense()` — true when the observed object creation rate is high.
- `isLargeGrid()` — true when the map appears to exceed roughly 60 cells on a side.

These signals are estimated from observations rather than hardcoded to a specific configuration, so the same agents adapt correctly to both Config 1 and Config 2 without any parameter changes.

Communication Protocol

Agents communicate via a global broadcast channel. Because ordering 2 runs agents sequentially, a broadcast from agent N reaches later-ordered agents within the same step and earlier-ordered agents by the following step at the latest; the full team has the message within one step. The base class parses all incoming messages and stores results in shared protected fields, so specialist agents consume parsed data directly without writing any parsing code themselves.

Table 1 lists the nine message types.

Communication improves team performance in four ways. First, **faster discovery sharing**: fuel station discovery by any one agent immediately benefits all five others. Second, **deduplication**: CLAIM messages prevent two agents from wasting movement on the same target. Third, **freshness control**: EXPIRING warnings let all agents avoid routing toward targets that are about to vanish. Fourth, **coverage coordination**: HOTSPOT and SWAP messages guide exploration toward productive and uncovered regions.

Specialist Agents

Each specialized agent receives access to all global resources from the base infrastructure — A*, Planning, Memory, and Broadcast — and adds an additional decision-making layer at the top of the `think()` loop that is specific to its assigned role. Within each specialization, execution follows three levels of priority. First, `fuelSafetyOverride()` is executed to perform fuel-related actions whenever necessary. Next, `opportunisticAction()` is triggered to identify opportunities to pick up or drop an item at zero cost. If the

Type	Format	Purpose
FUEL	x,y	Fuel station location
TILE	x,y,time	New tile sighting
HOLE	x,y,time	New hole sighting
GONE	x,y	Object disappeared
CLAIM	x,y	Agent is targeting this cell
LOW	agentId,fuel	Low-fuel warning
EXPIRING	x,y,reason	Target likely to expire
HOTSPOT	x,y,density	High-density area
SWAP	agentId,status	Zone exploration complete

Table 1: The nine message types and their payloads.

agent is standing on a tile or hole at the beginning of its turn, one of these actions will be performed. After `opportunisticAction()` has executed, each specialized agent uses the remainder of its turn to carry out its own role-specific game logic.

FuelScoutAgent

`FuelScoutAgent` was designed to resolve the greatest risk in early gameplay, which is the fueling station never being discovered during this time frame. Therefore, the team members are forced to use an extremely conservative routing strategy, such as limiting their ability to explore.

Upon start-up, the `FuelScoutAgent` moves towards the center of the map. This choice of moving towards the center is done intentionally to allow for maximum sensor coverage in each of the four directions, thus providing the largest opportunity for early detection. Once the agent reaches the center of the map, it changes to a commit-based Least Visited Search algorithm with a radius of up to 20 cells. As the fuel level decreases, the effective radius will scale downward (`effectiveRadius = MAX_EXPLORE_RADIUS * fuelFraction`). Therefore, as the agent's margins decrease, it will become more conservative in its actions. The commitment length can be adjusted between 8 and 24 steps depending upon how far the agent is from its target, reducing thrashing due to constantly changing target locations. The fuel floor is dynamically computed as `xDim + yDim + safetyBuffer`, representing the worst-case travel distance rather than a hardcoded value.

Once the station has been located and announced, if the fuel level is below 60% the agent will refuel and fall through to `super.think()`, acting as a general-purpose helper for the rest of the simulation. During the search phase, it also broadcasts `LOW` warnings whenever fuel drops below 1.2× `baseFloor`, throttled to no more than once every 4 steps, enabling teammates to route away from fuel-scarce regions.

ExplorerAgent

The purpose of the `ExplorerAgent` is to ensure the map is uniformly explored through six vertical strips along the x-axis of the map. When the `ExplorerAgent` initializes, each agent is assigned one zone of responsibility, thereby avoiding redundant exploration of the same region by multiple agents.

To perform efficient pathfinding within its zone, the agent utilizes `getLeastVisitedTarget()` with a varying radius depending on whether it is running on a large or small grid (`isLargeGrid() ? 20 : 10`) to find the most outdated observation timestamp, essentially the area it has not visited for the longest time.

In addition to timestamp-driven navigation, if there are any `HOTSPOT`s in the agent's zone beyond a certain threshold (either 15 or 8 depending on grid size), it will utilize `HOTSPOT` positioning. However, these thresholds are used to prevent the agent from leaving its zone for distant hotspots.

Once an agent reaches 80% coverage of its zone, as determined by the number of cells within the agent's zone that have been visited during the current simulation cycle, it sends out a `SWAP` message indicating completion of its zone. At this point, it ceases to use `ExplorerAgent` behaviour and defaults to base agent behaviour, collecting and delivering tiles for the remainder of the simulation.

TileHunterAgent

`TileHunterAgent` focuses on efficient tile collection through adaptive batching and conflict-free claiming. Its primary behavioural parameter is the tile target: in stable, long-lifetime environments (Config 1) it batches up to three tiles before delivering, amortising the fuel cost of a delivery trip over three reward points; in short-lifetime environments (Config 2) it delivers after every single tile to avoid holes expiring while it is still collecting. This adaptive strategy is necessary, if we perform batching in a volatile environment we risk the agent arriving at a hole that no longer exists, while delivering one tile at a time in a stable environment wastes fuel on unnecessary round trips.

To maximise efficiency, when a tile target is identified, `TileHunterAgent` immediately claims it in shared memory before moving, ensuring no teammate independently selects the same target and duplicates the trip. Before committing to any target, it also applies a reachability check via `isReachableBeforeExpiry()`, comparing the Manhattan distance to the target against its remaining lifetime using environment-adaptive thresholds (0.65 for

short-lifetime, 0.85 for long-lifetime environments). This prevents the agent from chasing tiles or holes that were observed earlier in memory but are unlikely to still exist on arrival. The thresholds were tightened from an earlier 0.7/0.9 configuration after observing that agents were committing to targets near the end of their lifetime and arriving to find them gone.

When no viable tile is known in memory, the agent does not fall back immediately to a blind lawnmower sweep. Instead, it first checks for `HOTSPOT` broadcasts from the `DeliveryOptimizerAgent` and navigates toward the nearest known tile-dense cluster. This biases exploration toward productive regions of the grid. Only if no hotspot is known does it revert to the base agent's systematic lawnmower exploration pattern.

HoleFillerAgent

`HoleFillerAgent` serves a dual role: it is both a delivery agent and a team-wide warning system against wasted trips. Before committing to any hole, it runs two sequential validity checks. First, it consults the shared `expiringTargets` list populated by teammate `EXPIRING` broadcasts; if the candidate hole appears on this list, the plan is voided immediately and the agent seeks an alternative. Second, it independently projects whether the hole will still exist on arrival by comparing the Manhattan distance to the target against the hole's remaining lifetime (current time minus observation timestamp subtracted from the total configured lifetime). The threshold applied to this projection adapts to environment volatility: 0.65 in short-lifetime environments and 0.85 in long-lifetime ones. If the projected travel cost exceeds the threshold fraction of remaining lifetime, the hole is considered unreachable and the plan is abandoned.

Critically, when `HoleFillerAgent` detects an imminent expiry through its own projection, it does not simply replan silently. It broadcasts an `EXPIRING` message to the entire team, ensuring that any other agent currently committed to the same hole also voids its plan simultaneously. This propagation prevents scenarios where multiple agents independently arrive at the same expired location, each having wasted the fuel for the trip.

To maintain team-wide hole awareness beyond individual sensor ranges, `HoleFillerAgent` periodically rebroadcasts known hole positions from its memory. The rebroadcast interval is adaptive: every 10 simulation steps in short-lifetime environments (Config 2) and every 15 steps in long-lifetime environments (Config 1). The freshness (age penalty) cutoff applied before rebroadcasting is equally adaptive, only holes younger than 0.5× their total lifetime are rebroadcast in volatile environments, and 0.7× in stable ones. This ensures that teammates receive only actionable hole information rather than stale entries that are unlikely to still exist, reducing noise in the shared message channel.

DeliveryOptimizerAgent

`DeliveryOptimizerAgent` maximises reward per fuel unit by replacing simple nearest-hole routing with a composite scoring function and structured decision cascade. Its `think()` method implements seven sequential gates, from fuel emergency handling through tile hunting, en-route batching, delivery, and hotspot navigation, with each gate only activating if higher-priority conditions are unmet.

When selecting a delivery target, the agent scores all known unclaimed holes using the following function:

$$\text{score}(h) = d(a,h) - w_b \cdot \text{bonus}(h) + w_f \cdot \frac{\text{age}(h)}{\text{lifetime}} \quad (1)$$

where $d(a,h)$ is the Manhattan distance from the agent to hole h, $\text{bonus}(h)$ is a tile proximity bonus computed as the sum of $1/(1+d(h,t))$ over all tiles t within a cluster radius of h, and $\text{age}(h)/\text{lifetime}$ is the fractional age of the hole penalising stale targets. The weights w_b (bonus weight) and cluster radius adapt to environment density: 4.5 and 8 cells in dense environments, 1.5 and 12 cells in sparse ones. The freshness penalty weight w_f was reduced from 15.0 to 10.0 after observing that the original value over-penalised slightly older but perfectly reachable nearby holes, pushing the agent toward distant fresh targets unnecessarily. The dual-threshold expiry projection already filters truly unreachable holes, so the freshness penalty serves only as a soft preference rather than a hard filter.

Before scoring, each candidate hole is validated against `EXPIRING` broadcasts, arrival-time projection (thresholds 0.65/0.85), and fuel round-trip affordability. Holes failing any check are excluded from scoring entirely.

When carrying tiles but below the delivery threshold, the agent performs en-route batching: it identifies tiles whose collection adds at most a configurable detour budget beyond the direct path to the target hole. The budget is 2 steps in short-lifetime environments and 4 steps in long-lifetime ones, ensuring the agent never risks a hole expiring by taking too large a detour.

To prevent goal oscillation, where an agent switches targets every step as scores fluctuate, the agent commits to a selected hole for a number of steps proportional to the distance to that target, bounded between a minimum of 6 and a maximum of 18 steps. The commitment is released early only if an `EXPIRING` signal is received for the committed target or fuel becomes critical.

Moreover, `DeliveryOptimizerAgent` acts as the team's dedicated `HOTSPOT` broadcaster. It groups all remembered tiles into 7×7 spatial clusters, computes a centroid and density for each cluster containing at least two tiles, and broadcasts the two densest clusters as `HOTSPOT` messages. To reduce channel noise, broadcasts are throttled to once every five simulation steps.

SmarterReplanningAgent

`SmarterReplanningAgent` acts as the system's reliability expert by implementing the adaptive cautious agent

concept from (1), applying predictive plan validation before any movement is executed. At each step, before acting on its current plan, it runs three sequential checks on the active goal.

The first check consults the shared `expiringTargets` list. If any teammate has already broadcast an `EXPIRING` signal for the current goal, the plan is aborted immediately without further computation, since the target has already been identified as unreachable by another agent, avoiding redundancy.

The second check is an independent arrival-time projection. The agent computes the Manhattan distance to the goal and compares it against the goal's estimated remaining lifetime derived from its observation timestamp. The expiry threshold adapts to environment volatility: 0.65 in short-lifetime environments (Config 2) and 0.85 in long-lifetime ones (Config 1). These thresholds were tightened from an original 0.7/0.9 configuration to trigger earlier plan abandonment, which proved especially important in Config 2 where targets expire within 30 steps. If the check fails, the agent broadcasts an `EXPIRING` warning before replanning, propagating the detection to teammates who may be pursuing the same goal.

The third check verifies fuel round-trip affordability. The agent estimates the total fuel cost of reaching the goal and returning to the known fuel station, adding a safety margin that scales with grid size (`max(50, gridWidth / 4)`). If current fuel falls below this estimate, the plan is voided and the agent navigates directly to the fuel station, prioritising survival over the current delivery. This check prevents the failure mode observed in earlier versions, where agents would commit to a distant goal, run out of fuel mid-path, and contribute nothing for the remainder of the simulation.

The agent then executes its planned move only if all three checks pass. This conservative posture trades some aggressiveness, the agent will abandon goals that a less cautious agent might successfully complete for reliability, virtually eliminating fuel deaths from overcommitment and reducing the number of wasted trips the team collectively makes to expired targets.

Experimental Configurations

The system was evaluated on two configurations derived from the MASON TileWorld parameters. Table 2 summarises their key differences.

Config 1 is the easier environment: objects are sparse but long-lived, so careful planning and commitment pay off. Config 2 is harder: the larger grid puts more pressure on fuel, and the short object lifetime means memory information becomes stale quickly, increasing the cost of a bad plan.

Results

Phase Development (Config 1)

Table 3 shows the average reward achieved at each development phase, measured on Config 1. Each phase added a distinct capability.

Phase 1 established a survival baseline: `SmartTWAgent` and `SmartTWAgentMemory` replaced the default agent with fuel-aware movement, memory decay, and opportunistic tile pickup. The system reacted to local events but could not yet route toward high-value work.

Phase 2 added `SmartTWPlanner` with A* pathfinding. Agents stopped wandering and began executing goal-directed plans. The first major reward jump followed because the same map knowledge was now exploited efficiently.

Phase 3 introduced multi-agent coordination: zone-based exploration, message broadcasts, shared tile and hole knowledge, and claim-based deconfliction. The score jump from Phase 2 to Phase 3 (160.3 to 541.5, a 3.4× increase) was the largest in the project and came from structural rather than incremental improvement: six agents began working as a team rather than as independent individuals.

Phase 4 unfolded in three steps. First, adaptive improvements to `SmartTWAgent` and `SmartTWAgentMemory` directly: runtime environment detection, adaptive fuel safety margins, and smarter planning heuristics. Second, a two-stage architectural transition: the uniform agent loop was first broken into individually instantiated person-named subclasses (one per team member), then renamed to role-named classes (`TileHunterAgent`, `HoleFillerAgent`, etc.), both stages initially delegating entirely to the shared base. Third, follow-up commits added the substantive role-specific logic — tighter expiry thresholds, freshness-aware delivery scoring, reachability checks, and throttled broadcasts. The 864.3 average reflects this full three-step progression; the largest gains came from the adaptive base improvements and the coordination already established in Phase 3, with role-specific behaviour providing targeted quality refinements on top.

Final Validation (Phase 4)

Table 4 shows the Phase 4 validation results on both configurations.

Both 10-run validations completed with zero failures. Config 2 returned a slightly higher average (888.6 vs. 864.3), but Config 1 remains the clearest success case because its environment is more amenable to the coordination and planning strategies the system relies on. Config 2 structural risks are discussed in the analysis below.

Analysis

Strengths

Shared-base scalability. Implementing pathfinding, memory, and communication once in the base class means every algorithmic improvement benefits all six agents simultaneously. This was the most important engineering decision in the project.

Coordination under partial observability. Without communication, each agent is limited to what it

Parameter	Config 1	Config 2
Grid size	50 × 50	80 × 80
Tile rate (per step)	0.2	2.0
Hole rate (per step)	0.2	2.0
Obstacle rate (per step)	0.2	2.0
Object lifetime (steps)	100	30
Agent fuel	500	500

Table 2: Parameters for Config 1 and Config 2.

Phase	Avg	Min	Max	Key addition
Phase 1	32.1	0	55	Survival and memory
Phase 2	160.3	0	249	A* planning, delivery
Phase 3	541.5	469	601	Coordination, communication
Phase 4	864.3	801	959	Adaptive base + role specialisation

Table 3: Config 1 average reward by development phase. Phases 1–3 are frozen historical measurements from earlier codebase states; Phase 4 is a current rerun on the active codebase after all follow-up commits.

Config	Avg reward	Final reward	Failures
Config 1	864.3	959	0
Config 2	888.6	981	0

Table 4: Phase 4 validation results. Each config was evaluated with `TileworldMain` running 10 independent random-seed trials (`iteration = 10`; see `TileworldMain.java:30`). Average reward is the mean final score across the 10 trials; Final reward is the score of the last trial. A failure is a trial whose final reward was zero. All three aggregate metrics are recorded in `MULTI_RUN_RESULTS.md`. Per-trial variance (individual min/max across the 10 trials) was not separately recorded for Config 2.

can directly observe. The broadcast protocol converts six local observers into a distributed shared-intelligence system. Claims prevent duplicated effort; EXPIRING warnings stop wasted movement; HOTSPOT messages direct exploration toward productive regions.

Measured performance trajectory. The phase scores confirm that each architectural addition produced a measurable gain. The largest gain (Phase 3 coordination) accounts for the majority of total improvement, validating the design priority of getting coordination right before tuning specialisation.

Cross-agent idea borrowing. Several mechanisms were refined across agents. For example, `DeliveryOptimizerAgent` explicitly combines expiry projection (originally developed for `SmarterReplanningAgent`), hotspot routing, and freshness-aware scoring into a single delivery policy. Good ideas spread through the team incrementally.

Weaknesses

Config 2 structural risk. The 10-run validation produced a positive average reward (888.6), but the environment's structure creates latent reliability risk not fully captured by the aggregate. The 80×80 grid creates a larger search space for the fuel station; delayed discovery forces the whole team into conservative routing. Short object lifetimes increase the chance that remembered targets expire before an agent arrives, even with predictive expiry checking. Per-trial variance (individual min/max across the 10 trials) was not separately recorded for Config 2, so the spread of outcomes is unquantified.

Early-run lifetime detection. The runtime environment classifier requires enough observed disappearances to estimate object lifetime accurately. In the first hundred steps of a Config 2 run, agents may still believe they are in a stable environment and apply overly generous planning horizons.

Fuel station discovery bottleneck. Many decisions across the system depend on the fuel station being found early. On a large map with an unlucky starting layout, delayed discovery creates cascading conservative behaviour across all agents until the station is located.

Coordination Tradeoff

The fundamental tradeoff in any multi-agent system is between the parallelism benefit — six agents covering the map simultaneously, collecting tiles, and filling holes in parallel — and the coordination cost of communication, claim management, and zone assignment overhead. This system is strongest when the environment gives coordination time to pay off: Config 1, with its stable, sparse objects, rewards careful planning and reliable delivery. Config 2 tests the limits because the environment changes fast enough that shared information can become stale before the team acts on it.

Conclusion

We presented a six-agent TileWorld multi-agent system that improves from a survival baseline (Phase 1, avg. 32.1) to a coordinated specialist team (Phase 4, avg. 864.3 on Config 1) through four development phases. A 10-run Phase 4 validation on Config 2 achieved an average reward of 888.6; the four-phase development history is documented for Config 1 only. The key design insight is the two-layer separation between shared infrastructure and role specialisation: improvements to the

base layer compound across all agents, while specialist layers add targeted decision quality without duplicating infrastructure.

The most impactful single change was the introduction of multi-agent coordination in Phase 3, confirming the importance of communication and deconfliction over individual agent optimisation. Phase 4 role specialisation added further gains by covering different failure modes of the task: fuel discovery, tile collection efficiency, delivery reliability, map coverage, high-value routing, and predictive failure prevention.

Future work could improve Config 2 reliability through two main directions: earlier fuel station discovery via a dedicated fast-scout strategy that temporarily overrides zone assignments at the start of a run, and adaptive lifetime thresholds that update more aggressively in the first 50 steps based on early disappearance observations.

Disclosure on the use of AI

Through our this project Gemini 3.1 and Claude 4.6 Sonnet were used to correct grammatical and phrasing errors, and format the citations. Claude 4.6 Sonnet was also used to help with coding related tasks, especially with Java . Other than this, no AI system was used during the preparation of this report, and it mirrors the group's understanding of the concepts presented.

References

[1] Pollack, M. E., and Ringuette, M. 1990. Introducing the Tileworld: Experimentally evaluating agent architectures. In *Proceedings of the Eighth National Conference on Artificial Intelligence (AAAI-90)*, 183–189.

[2] Kinny, D., and Georgeff, M. 1991. Commitment and effectiveness of situated agents. In *Proceedings of the Twelfth International Joint Conference on Artificial Intelligence (IJCAI-91)*, 82–88.